

Details_Explanation_In_Bangla

AI Lab B: সব সমস্যার বিস্তারিত ব্যাখ্যা

এই ফাইলে `AI Lab B/problems.md`-এ থাকা ৮টি সমস্যার ধারণা, কাজের নিয়ম এবং Python প্রোগ্রামের মূল যুক্তি বাংলায় ব্যাখ্যা করা হয়েছে।

১. 4-Queen Problem

সমস্যা কী?

4-Queen Problem হলো N-Queen সমস্যার একটি ছোট সংস্করণ। এখানে ৪ x ৪ দাবার বোর্ডে ৪টি Queen বসাতে হবে, যেন কোনো Queen অন্য কোনো Queen-কে আক্রমণ করতে না পারে। দাবার Queen একই row, same column এবং diagonal-এ আক্রমণ করতে পারে। তাই প্রতিটি Queen এমনভাবে বসাতে হবে যেন:

- একই row-তে একাধিক Queen না থাকে
- একই column-এ একাধিক Queen না থাকে
- একই diagonal-এ একাধিক Queen না থাকে

মূল ধারণা

এই সমস্যাটি সাধারণত Backtracking ব্যবহার করে সমাধান করা হয়। Backtracking হলো এমন একটি পদ্ধতি যেখানে একটি সম্ভাব্য সিদ্ধান্ত নেওয়া হয়, তারপর সেটি ভুল হলে আগের অবস্থায় ফিরে গিয়ে অন্য সিদ্ধান্ত পরীক্ষা করা হয়।

Algorithm

1. প্রথম row থেকে Queen বসানো শুরু করা হয়।
2. প্রতিটি column পরীক্ষা করা হয়।
3. যদি কোনো position নিরাপদ হয়, সেখানে Queen বসানো হয়।
4. এরপর পরের row-তে যাওয়া হয়।
5. কোনো row-তে Queen বসানোর নিরাপদ জায়গা না পাওয়া গেলে আগের row-তে ফিরে যাওয়া হয়।

6. সব row-তে Queen বসানো গেলে সমাধান পাওয়া যায়।

Code-এর কাজ

`is_safe(row, col)` function পরীক্ষা করে নির্দিষ্ট position-এ Queen বসানো নিরাপদ কি না।

`solve(row)` function recursive ভাবে এক row থেকে পরের row-তে Queen বসায়। যদি কোনো অবস্থায় সমস্যা হয়, তাহলে Queen সরিয়ে আবার অন্য column চেষ্টা করে।

Output

প্রোগ্রাম একটি valid board দেখায়, যেমন:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

২. 8-Puzzle Problem

সমস্যা কী?

8-Puzzle হলো ৩ x ৩ grid-এর একটি puzzle। এতে ১ থেকে ৮ পর্যন্ত tile থাকে এবং একটি খালি ঘর থাকে, যাকে প্রোগ্রামে ০ দিয়ে বোঝানো হয়েছে।

লক্ষ্য হলো start state থেকে goal state-এ পৌঁছানো। খালি ঘরটি উপর, নিচ, বাম বা ডানে সরিয়ে puzzle solve করা হয়।

উদাহরণ

Start state:

```
1 2 3  
4 0 6  
7 5 8
```

Goal state:

```
1 2 3  
4 5 6
```

মূল ধারণা

এই সমস্যার solution-এ Breadth First Search বা BFS ব্যবহার করা হয়েছে। BFS সবচেয়ে কম move-এ solution খুঁজে বের করতে পারে, কারণ এটি level by level state পরীক্ষা করে।

Algorithm

1. Start state queue-তে রাখা হয়।
2. Queue থেকে একটি state নেওয়া হয়।
3. যদি state goal হয়, solution path return করা হয়।
4. নাহলে খালি ঘর সরিয়ে নতুন state তৈরি করা হয়।
5. নতুন state আগে দেখা না হলে queue-তে রাখা হয়।
6. Goal না পাওয়া পর্যন্ত প্রক্রিয়া চলতে থাকে।

Code-এর কাজ

`get_neighbors(state)` function current state থেকে possible next states তৈরি করে।

`solve_puzzle(start, goal)` function BFS চালিয়ে goal state খুঁজে বের করে।

`print_board(state)` function puzzle board সুন্দরভাবে print করে।

৩. Tower of Hanoi Problem

সমস্যা কী?

Tower of Hanoi একটি classic recursion problem। এখানে তিনটি rod থাকে:

- Source rod
- Auxiliary rod
- Destination rod

কিছু disk source rod-এ রাখা থাকে। লক্ষ্য হলো সব disk destination rod-এ নেওয়া।

নিয়ম

1. একবারে শুধু একটি disk সরানো যাবে।
2. বড় disk কখনো ছোট disk-এর উপর রাখা যাবে না।
3. Auxiliary rod ব্যবহার করা যাবে।

মূল ধারণা

Tower of Hanoi recursion দিয়ে সহজে solve করা যায়।

যদি n disk থাকে:

1. প্রথমে $n - 1$ disk source থেকে auxiliary rod-এ সরাও।
2. সবচেয়ে বড় disk source থেকে destination rod-এ সরাও।
3. এরপর $n - 1$ disk auxiliary থেকে destination rod-এ সরাও।

Algorithm

1. যদি disk সংখ্যা ১ হয়, সরাসরি source থেকে destination-এ move করা হয়।
2. নাহলে recursive call করে ছোট disk-গুলো সরানো হয়।
3. বড় disk move করা হয়।
4. আবার recursive call করে auxiliary থেকে destination-এ disk সরানো হয়।

Code-এর কাজ

`tower_of_hanoi(n, source, auxiliary, destination)` function প্রতিটি disk move print করে।
User input থেকে disk সংখ্যা নেওয়া হয়।

8. Traveling Salesman Problem

সমস্যা কী?

Traveling Salesman Problem বা TSP-তে একজন salesman কয়েকটি city ভ্রমণ করবে।
তাকে প্রতিটি city একবার করে visit করে আবার starting city-তে ফিরে আসতে হবে।

লক্ষ্য হলো total travel cost বা distance সবচেয়ে কম করা।

মূল ধারণা

এই solution-এ brute force পদ্ধতি ব্যবহার করা হয়েছে। অর্থাৎ সব possible route পরীক্ষা করা হয়েছে এবং যার cost সবচেয়ে কম, সেটিকে best path ধরা হয়েছে।

Algorithm

1. Starting city fixed রাখা হয়।
2. বাকি city-গুলোর সব permutation তৈরি করা হয়।
3. প্রতিটি route-এর total cost হিসাব করা হয়।
4. সবচেয়ে কম cost-এর route সংরক্ষণ করা হয়।
5. শেষে best path এবং minimum cost print করা হয়।

Code-এর কাজ

`graph` variable-এ city থেকে city-তে যাওয়ার cost matrix রাখা হয়েছে।

`permutations(cities)` দিয়ে সব possible order তৈরি করা হয়েছে।

প্রতিটি path-এর cost হিসাব করে minimum cost বের করা হয়েছে।

সীমাবদ্ধতা

Brute force পদ্ধতি ছোট input-এর জন্য ভালো। কিন্তু city সংখ্যা বেশি হলে route সংখ্যা অনেক বেড়ে যায়, তাই বড় problem-এর জন্য এটি efficient নয়।

৫. Depth First Search

সমস্যা কী?

Depth First Search বা DFS হলো graph traversal algorithm। এটি graph-এর node visit করার একটি পদ্ধতি।

DFS একটি node থেকে শুরু করে যত গভীরে যাওয়া যায় তত গভীরে যায়। এরপর আর যাওয়ার জায়গা না থাকলে backtrack করে।

মূল ধারণা

DFS সাধারণত recursion বা stack ব্যবহার করে implement করা হয়। এই solution-এ recursion ব্যবহার করা হয়েছে।

Algorithm

1. Start node visit করা হয়।
2. Node-টিকে visited set-এ রাখা হয়।
3. ঐ node-এর প্রতিটি neighbor পরীক্ষা করা হয়।
4. কোনো neighbor visited না হলে সেটিতে recursive ভাবে DFS চালানো হয়।
5. সব reachable node visit না হওয়া পর্যন্ত চলতে থাকে।

Code-এর কাজ

`graph` dictionary দিয়ে graph represent করা হয়েছে।

`dfs(node, visited)` function current node print করে এবং তার unvisited neighbor-এ যায়।

Example Traversal

Start node `A` হলে traversal হতে পারে:

```
A B D E F C
```

৬. Breadth First Search

সমস্যা কী?

Breadth First Search বা BFS হলো আরেকটি graph traversal algorithm। এটি graph-এর node level by level visit করে।

DFS যেখানে গভীরে যায়, BFS সেখানে প্রথমে start node-এর সব nearest neighbor visit করে, তারপর পরের level-এ যায়।

মূল ধারণা

BFS সাধারণত queue ব্যবহার করে implement করা হয়।

Algorithm

1. Start node queue-তে রাখা হয়।
2. Start node visited হিসেবে mark করা হয়।

3. Queue থেকে প্রথম node বের করা হয়।
4. Node print করা হয়।
5. তার unvisited neighbor-গুলো queue-তে যোগ করা হয়।
6. Queue empty না হওয়া পর্যন্ত চলতে থাকে।

Code-এর কাজ

`deque` ব্যবহার করে queue তৈরি করা হয়েছে।

`bfs(start)` function start node থেকে BFS traversal চালায়।

Example Traversal

Start node `A` হলে traversal হতে পারে:

A B C D E F

৭. Hill Climbing Algorithm

সমস্যা কী?

Hill Climbing একটি optimization algorithm। এর কাজ হলো কোনো objective function-এর maximum বা minimum value খুঁজে বের করা।

এই প্রোগ্রামে function হলো:

$$f(x) = -(x - 3)^2 + 10$$

এই function-এর maximum value পাওয়া যায় যখন `x = 3`।

মূল ধারণা

Hill Climbing algorithm current position থেকে পাশের position পরীক্ষা করে। যদি পাশের position ভালো হয়, তাহলে সেখানে যায়। আর যদি কোনো পাশের position ভালো না হয়, তাহলে থেমে যায়।

Algorithm

1. একটি initial value নেওয়া হয়।
2. Current value-এর left এবং right neighbor পরীক্ষা করা হয়।
3. যে neighbor-এর objective value বেশি, সেখানে move করা হয়।
4. যদি কোনো neighbor current value থেকে ভালো না হয়, algorithm থেমে যায়।
5. Current value-কে best solution ধরা হয়।

Code-এর কাজ

`objective_function(x)` function কোনো `x`-এর value হিসাব করে।

`current` variable current position রাখে।

Loop চলতে থাকে যতক্ষণ পর্যন্ত ভালো neighbor পাওয়া যায়।

সীমাবদ্ধতা

Hill Climbing local maximum-এ আটকে যেতে পারে। তাই সবসময় global maximum পাওয়া যাবে এমন নিশ্চয়তা নেই। তবে এই simple function-এর ক্ষেত্রে এটি সঠিক answer দেয়।

৮. Sort a List

সমস্যা কী?

এই সমস্যায় একটি list-এর elements ছোট থেকে বড় ক্রমে সাজাতে হবে।

মূল ধারণা

Python-এ list sort করার জন্য built-in `sort()` method আছে। এটি list-এর elements ascending order-এ সাজায়।

Algorithm

1. User থেকে numbers input নেওয়া হয়।
2. Input string split করে আলাদা আলাদা number বানানো হয়।
3. প্রতিটি value `float` এ convert করা হয়।
4. `sort()` method দিয়ে list sort করা হয়।
5. Sorted list print করা হয়।

Code-এর কাজ

```
numbers = input("Enter numbers separated by spaces: ").split()  
numbers = [float(number) for number in numbers]  
numbers.sort()
```

এই অংশে input list নেওয়া, number-এ convert করা এবং sort করা হয়েছে।

Example

Input:

5 2 9 1 3

Output:

Sorted list: [1.0, 2.0, 3.0, 5.0, 9.0]

সংক্ষিপ্ত তুলনা

Problem	Main Technique
4-Queen	Backtracking
8-Puzzle	Breadth First Search
Tower of Hanoi	Recursion
Traveling Salesman	Brute Force / Permutation
DFS	Recursion / Stack concept
BFS	Queue
Hill Climbing	Optimization
Sort List	Built-in sorting